

TriageDesk: AI Assistant for SOC Alert Triage and Log Analysis

EECS 3311 Fall 2026, Course Project Stage 1: Project Design Report

Student: Hoang Vinh Nguyen, student number: 219420512

GitHub repository: <https://github.com/hvinhnguyen08/EECS3311-Project-TriageDesk/>

1. Project Overview

1.1 Problem and Motivation

A SOC analyst spends most of a shift on the same loop: open an alert, read the log lines behind it, check the IP address or file hash on threat-intelligence sites, look for related activity on the same host, decide whether it is a real attack, and write it up. The work is repetitive and spread across many tools. A SIEM such as Wazuh can raise hundreds of alerts a day, many of them duplicates of the same event (one SSH brute-force attempt can produce hundreds of "authentication failed" alerts).

TriageDesk imports Wazuh alerts and raw logs, groups related detections into a short, prioritised list of cases, and gives the analyst an AI agent that investigates each case with real tools, proposes a verdict with its reasoning, suggests response steps and drafts the incident report. The analyst makes every final decision.

1.2 Target Users

- **L1 SOC analysts** (primary users): import the day's alerts and logs, work the case list, close false positives and escalate real incidents.
- **L2 analysts and incident responders**: receive escalated cases with the enrichment, ATT&CK mapping and report already prepared.
- **Students and small IT teams** without a full SIEM: analyse Linux or web-server logs and learn how an investigation is done.

1.3 Agent Description

What the agent can do. The agent (SocAgent) receives a task, decides which tools it needs, calls them, reads the results and then answers. Its tools are:

- **search_events**: find events matching a host, user, IP, text or time range.
- **count_by**: count events grouped by a field (for example failed logins per source IP).
- **lookup_ioc**: get the reputation of an IP, domain or hash from threat-intelligence services.

With these tools the agent triages a case (F06), summarises an imported log (F04), suggests ATT&CK techniques when the rules give none (F08), recommends response steps (F10), writes the report narrative (F11) and answers free-form questions about the data (F12).

Why an AI agent is appropriate. The investigation path is not fixed: whether to look up an IP, pull events from the same host or count failed logins depends on what has been found so far. A tool-calling agent chooses the next step from intermediate results, which a single LLM call or a fixed script cannot do. Summarising logs and writing explanations and reports are language tasks that LLMs do well. Every fact in an answer comes from a tool call on real data, and the tool calls are shown to the analyst, so the reasoning can be checked. Parsing, detection rules, case grouping, case status and the dashboard stay deterministic code.

1.4 AI/LLM Model(s)

- **Default:** an OpenAI chat model with function calling (a low-cost model; the model name is set in the configuration file).
- **Local option:** an open-weight model served by Ollama (for example Llama 3.1 8B), for organisations that cannot send log data to an external API. Ollama exposes the same OpenAI-compatible API, so no code changes are needed.
- **Mock:** MockLlmClient returns scripted replies so that tests and the demo run without an API key or network.

How the AI model interacts with the rest of the system. A controller or CLI command calls the SocAssistant facade, which loads the case or data and calls SocAgent. SocAgent sends the task and the list of allowed tools to the model through the LlmClient interface, which returns an LlmReply containing either a tool call or the final answer. When the model asks for a tool, SocAgent finds it in ToolRegistry, executes it (for example SearchEventsTool queries EventStore), and sends the result back to the model. This repeats until the model gives a final answer or the step limit (6) is reached. SocAgent then validates the answer and converts it into a domain object (TriageVerdict, ResponsePlan or AgentResult); invalid output becomes "needs review" instead of being trusted.

1.5 Overall Architecture

TriageDesk is a single Java 21 application organised in layers:

- **Presentation:** a JavaFX GUI (views and controllers) and a picocli command-line interface. Both call only the SocAssistant facade.
- **Facade:** SocAssistant, one method per feature, shared by the GUI and CLI.
- **Services:** log ingestion (LogParserFactory, parsers, EventStore), detection (DetectionEngine and detectors), case management (CaseManager), threat-intel enrichment (EnrichmentService and adapters), ATT&CK mapping (AttackMapper) and reporting (ReportGenerator).
- **AI agent:** SocAgent, ToolRegistry and its tools, and the LlmClient implementations.
- **Storage:** events in memory and in a data folder (so separate CLI commands share them), cases as JSON files, reports as Markdown files, and a bundled copy of the MITRE ATT&CK catalogue.
- **External services:** the LLM API, AbuseIPDB and VirusTotal, reached over HTTPS.

GUI screens: Main (import, detections, case list, log summary), Case (verdict, IOCs, techniques, notes, response plan, report), Dashboard, Ask the Agent. CLI commands: import, detect, summarize, enrich, map, triage, status, note, respond, report, ask, stats.

2. Detailed Feature Specifications

The project has 13 features. Their AI involvement is summarised below and specified in detail after the table.

ID	Feature	AI involvement
F01	Import Wazuh Alerts	Deterministic
F02	Import and Parse Raw Logs	Deterministic
F03	Rule-Based Log Detections	Deterministic
F04	AI Log Summary	AI
F05	Group Findings into Prioritised Cases	Deterministic
F06	AI Case Triage	AI
F07	IOC Extraction and Threat-Intel Enrichment	Deterministic
F08	MITRE ATT&CK Mapping	Hybrid
F09	Case Lifecycle Management	Deterministic
F10	AI Response Recommendations	AI
F11	Incident Report Generation	Hybrid
F12	Ask the Agent	AI
F13	Security Dashboard	Deterministic

F01: Import Wazuh Alerts

Feature ID and Name	F01: Import Wazuh Alerts
Description	Loads a Wazuh alerts.json export and converts each alert into a SecurityEvent that keeps the rule ID, rule level, host, source IP, user and the MITRE technique IDs Wazuh attached.
User Interaction	Main window, Import button, then choose the file. CLI: triagedesk import alerts.json
Input	Path to a Wazuh alerts.json file (one JSON alert per line).
Output	Events stored in EventStore; a status message with the number of events imported and skipped.
AI Involvement	Deterministic
Expected Workflow	The format is detected, the factory creates a WazuhAlertParser, each line is parsed into a SecurityEvent, and the events are added to EventStore.
Error/Alternative Cases	File missing or unreadable: error message, nothing imported. Unsupported format: "Unsupported file format". Malformed lines: skipped and counted, the import continues.

F02: Import and Parse Raw Logs

Feature ID and Name	F02: Import and Parse Raw Logs
Description	Loads raw text logs and parses each line into a normalised SecurityEvent (time, host, user, source IP, message). Supported formats: Linux auth.log (sshd, sudo) and Apache/Nginx access logs.
User Interaction	Same Import button as F01; the format is detected from the first lines of the file. CLI: triagedesk import auth.log
Input	Path to a log file.
Output	Normalised events stored in EventStore; a status message.
AI Involvement	Deterministic
Expected Workflow	Same as F01, except the factory returns an AuthLogParser or ApacheLogParser.
Error/Alternative Cases	Unparseable lines are skipped and counted. Syslog lines have no year, so the file's year is assumed. Unsupported format: error message.

F03: Rule-Based Log Detections

Feature ID and Name	F03: Rule-Based Log Detections
Description	Runs detection rules over the imported events and produces findings: Wazuh alert (level 7 or higher), SSH brute force (10 or more failed logins from one IP in 5 minutes), successful login after failures, and web directory scan (30 or more HTTP 404 responses from one IP in 2 minutes). Thresholds are configurable.
User Interaction	Main window, Run Detections button. CLI: triagedesk detect
Input	The events in EventStore.
Output	A list of findings (detector, title, severity, IP/host/user, technique IDs, evidence events), passed to F05.
AI Involvement	Deterministic
Expected Workflow	DetectionEngine calls detect() on every registered Detector and collects the findings.
Error/Alternative Cases	No events loaded: "Import a file first". No findings: "No suspicious activity". A detector that fails is skipped and the others still run.

F04: AI Log Summary

Feature ID and Name	F04: AI Log Summary
Description	Writes a plain-English summary of an imported file: what activity it contains, the busiest hosts, users and IPs, anything unusual, and what to look at first.
User Interaction	Main window, select an imported file, Summarize button; the summary

	opens in a side panel. CLI: triagedesk summarize <batchId>
Input	The batch ID of an imported file.
Output	Summary text and the list of tool calls the agent made.
AI Involvement	AI
Expected Workflow	The agent calls count_by and search_events (limited to that file) to gather statistics, then writes the summary.
Error/Alternative Cases	LLM unreachable: "AI service unavailable". Step limit reached: partial summary marked incomplete. Empty file: no LLM call.

F05: Group Findings into Prioritised Cases

Feature ID and Name	F05: Group Findings into Prioritised Cases
Description	Groups related findings (same source IP or host within 60 minutes) into one case, sets its severity to the highest among its findings and gives it a priority score for the case queue.
User Interaction	Runs automatically after Run Detections; the case list shows the new cases sorted by priority. CLI: triagedesk detect prints the new cases
Input	Findings from F03.
Output	New cases in state New, saved as JSON files and listed in the GUI.
AI Involvement	Deterministic
Expected Workflow	CaseManager groups the findings, creates a Case for each group, computes the priority and saves each case.
Error/Alternative Cases	A finding that matches an open case is added to that case instead of creating a duplicate. Save failure: the case stays in memory and a warning is shown.

F06: AI Case Triage

Feature ID and Name	F06: AI Case Triage
Description	The agent investigates a case and proposes a verdict (true positive, false positive, benign, or needs review) with a confidence value and reasoning. It gathers its own evidence with tools: related events, event counts and IOC reputation.
User Interaction	Case view, AI Triage button; a verdict panel shows the verdict, confidence, reasoning and the tool calls used. CLI: triagedesk triage <caseId>
Input	A case ID; the case's findings, events and IOCs are included in the prompt.
Output	A TriageVerdict stored on the case; a New case moves to Investigating.
AI Involvement	AI
Expected Workflow	The agent loop sends the case to the LLM with the allowed tools,

	executes each requested tool, returns the results to the LLM and repeats until a final JSON verdict or 6 steps.
Error/Alternative Cases	LLM unreachable or invalid key: error shown, case unchanged. Invalid JSON or step limit reached: verdict NEEDS_REVIEW with the raw text. A tool error is returned to the model as the tool result.

F07: IOC Extraction and Threat-Intel Enrichment

Feature ID and Name	F07: IOC Extraction and Threat-Intel Enrichment
Description	Extracts indicators (IP addresses, domains, URLs, file hashes) from a case's events and looks them up in AbuseIPDB and VirusTotal. Results are cached for 24 hours.
User Interaction	Case view, Enrich IOCs button; a table shows each indicator with its reputation score. CLI: triagedesk enrich <caseId>
Input	A case ID; API keys from environment variables.
Output	A list of loc objects with score and source, stored on the case.
AI Involvement	Deterministic
Expected Workflow	IOCs are extracted with regular expressions, each one is looked up with every provider that supports its type, and the case is saved.
Error/Alternative Cases	Missing API key: provider skipped. Timeout or rate limit: score marked unavailable, other providers continue. Private IPs are not sent to external services.

F08: MITRE ATT&CK Mapping

Feature ID and Name	F08: MITRE ATT&CK Mapping
Description	Links a case to MITRE ATT&CK techniques (ID, name, tactic). It first uses the technique IDs from Wazuh rules and detectors; if there are none, the agent suggests IDs and only IDs found in the bundled ATT&CK catalogue are kept.
User Interaction	Case view, Map to ATT&CK button; techniques are listed with their tactic. CLI: triagedesk map <caseId>
Input	A case ID; the bundled ATT&CK catalogue.
Output	A list of AttackTechnique objects stored on the case.
AI Involvement	Hybrid
Expected Workflow	Look up the known IDs in the catalogue; if none, ask the agent and validate its suggestions against the catalogue.
Error/Alternative Cases	Suggested IDs not in the catalogue are discarded. No IDs and AI unavailable: "No mapping available".

F09: Case Lifecycle Management

Feature ID and Name	F09: Case Lifecycle Management
Description	Moves a case through its lifecycle (New, Investigating, Escalated, Closed), supports closing as true or false positive, reopening, and adding notes. Every change is recorded in the case history and invalid transitions are refused.
User Interaction	Case view, status buttons (Start, Escalate, Close, Reopen) with a comment box, and a Notes tab. CLI: triagedesk status <caseId> escalate -m "reason" and triagedesk note <caseId> "text"
Input	Case ID, action, comment or note text.
Output	The updated case (new state, history entry, notes), saved and redisplayed.
AI Involvement	Deterministic
Expected Workflow	The case forwards the action to its current state object, which decides the next state.
Error/Alternative Cases	Action not allowed in the current state (for example escalating a closed case): error, case unchanged. Empty note or missing close comment: rejected.

F10: AI Response Recommendations

Feature ID and Name	F10: AI Response Recommendations
Description	Generates an ordered checklist of containment and remediation steps for a triaged case (for example "Block 203.0.113.7 at the firewall", "Reset the password for user deploy"). The steps are advice only; the analyst ticks them off.
User Interaction	Case view, Recommend Response button; a checklist appears. CLI: triagedesk respond <caseId>
Input	A case ID; the case must have a verdict.
Output	A ResponsePlan stored on the case.
AI Involvement	AI
Expected Workflow	The agent receives the verdict, IOCs and techniques and returns a JSON list of steps.
Error/Alternative Cases	No verdict yet: "Run AI Triage first". Verdict is false positive: a single "document and close" step. LLM unavailable: error.

F11: Incident Report Generation

Feature ID and Name	F11: Incident Report Generation
Description	Creates a Markdown incident report for a case. Case facts, timeline, IOCs, ATT&CK techniques, verdict, notes and the response checklist

	are filled in by code; the executive summary and analysis are written by the agent and labelled as AI-generated.
User Interaction	Case view, Generate Report button; the report is saved in the reports folder and its path is shown. CLI: triagedesk report <caseId> -o report.md
Input	A case ID and an output path.
Output	A Markdown report file.
AI Involvement	Hybrid
Expected Workflow	The agent writes the narrative, then ReportGenerator builds the full report and writes the file.
Error/Alternative Cases	LLM unavailable: the report is still produced with "[AI narrative unavailable]". Write error: "Could not write report".

F12: Ask the Agent

Feature ID and Name	F12: Ask the Agent
Description	A chat panel (and CLI command) where the analyst asks questions about the loaded data in plain language, for example "Which IPs failed SSH login more than 10 times today?". The agent answers by querying the data with its tools and shows which tools it used.
User Interaction	Ask the Agent tab, type a question, Ask button. CLI: triagedesk ask "question"
Input	A question in plain text.
Output	The answer and the list of tool calls.
AI Involvement	AI
Expected Workflow	The agent chooses tools (count_by, search_events, lookup_ioc), runs them on the loaded data and answers from the results.
Error/Alternative Cases	No data loaded: "Import a file first". Question unrelated to the data: the agent says it can only answer about the loaded data. Step limit: partial answer marked incomplete.

F13: Security Dashboard

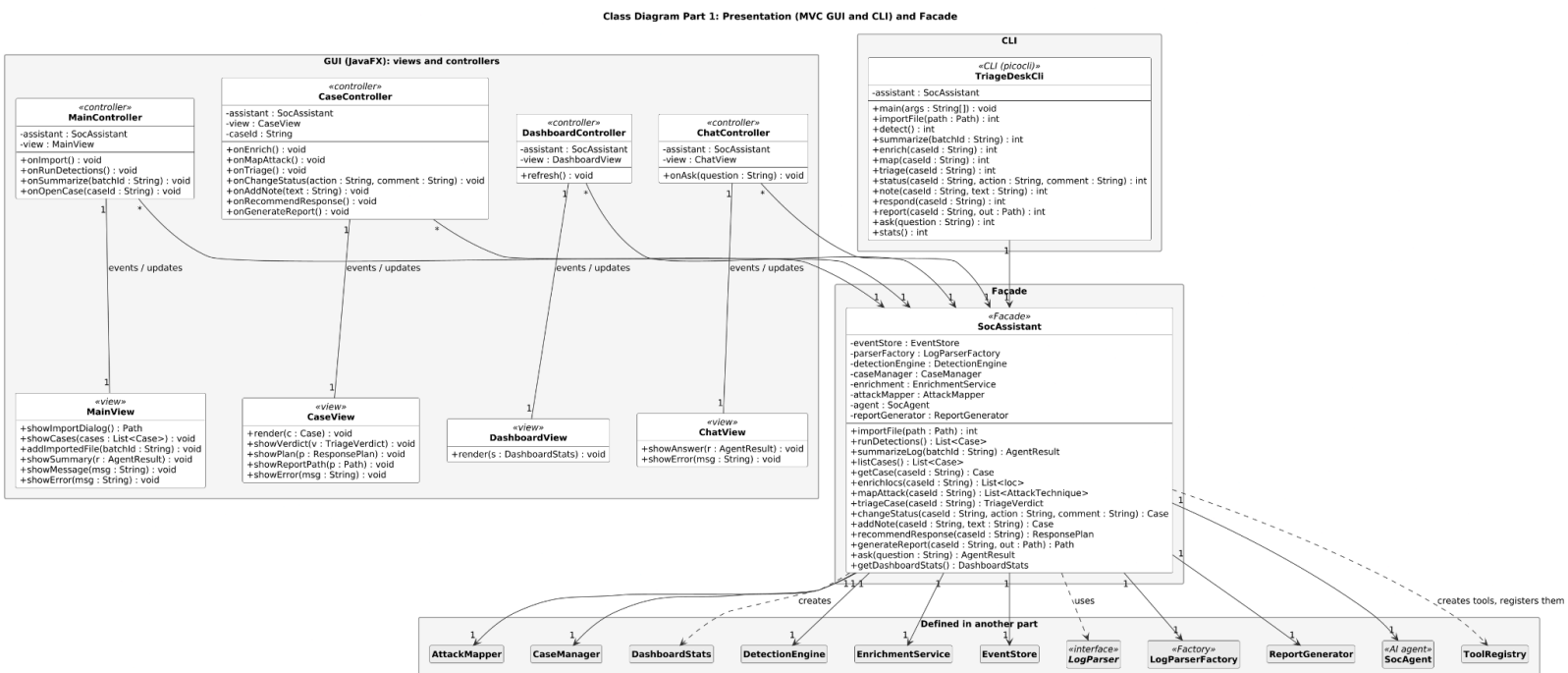
Feature ID and Name	F13: Security Dashboard
Description	Shows totals and charts for the loaded data and the case queue: total events, top source IPs, top users, and open cases by state.
User Interaction	Dashboard tab (default screen). CLI: triagedesk stats
Input	The contents of EventStore and CaseManager.
Output	DashboardStats rendered as charts (GUI) or text tables (CLI).
AI Involvement	Deterministic
Expected Workflow	The controller asks the facade for statistics and renders them.
Error/Alternative Cases	No data: empty dashboard with "Import a file to begin".

3. UML Class Diagram

The class diagram has 55 classes and interfaces. For readability it is shown in five parts; classes from another part appear as grey name-only boxes. It shows the traditional components (GUI, CLI, facade, services, domain) and the AI/agent components (SocAgent, LlmClient, ToolRegistry, tools), with attributes, methods, associations, dependencies, inheritance (generalisation and interface realisation), aggregation/composition and multiplicities.

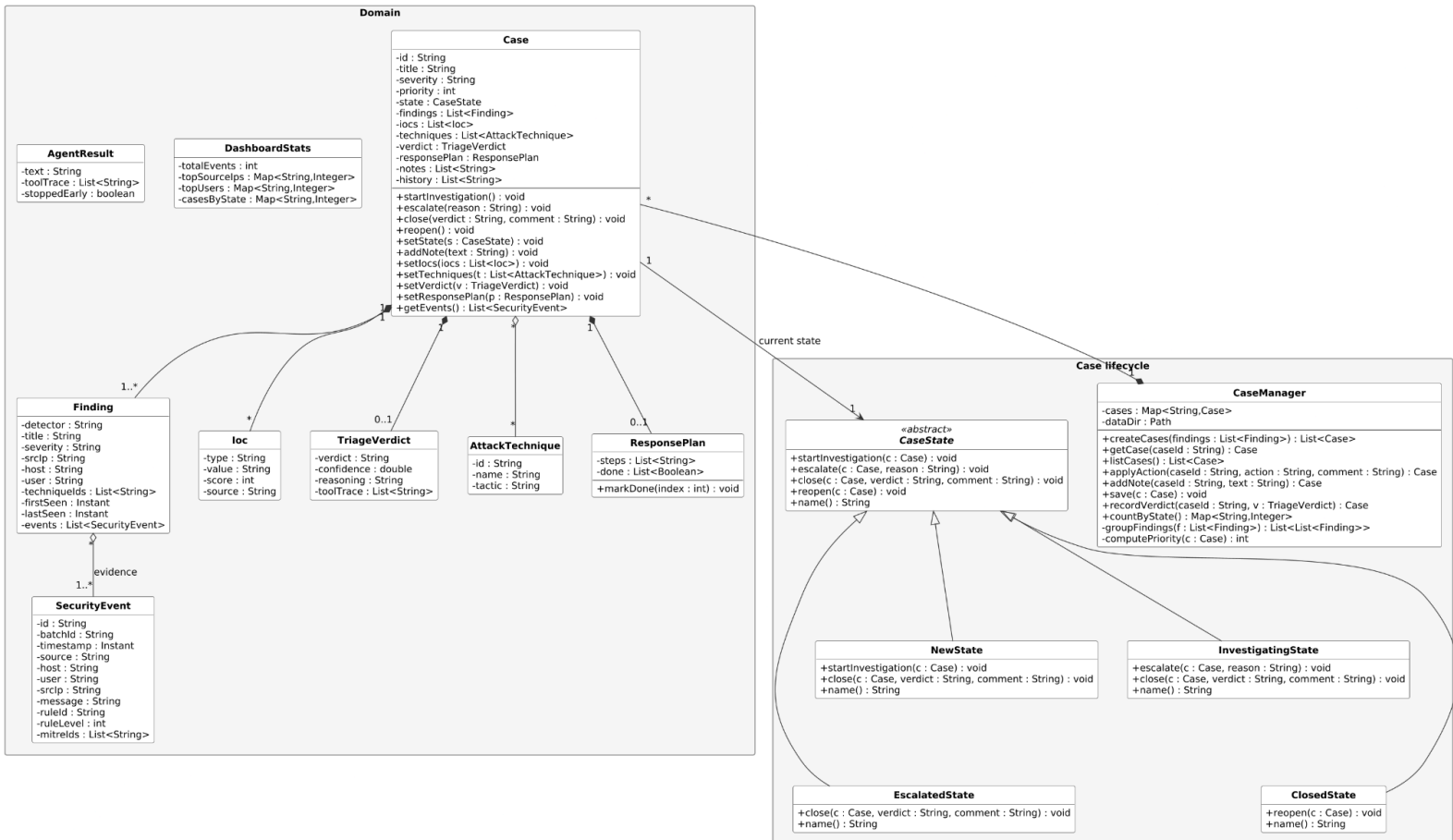
Part	Contents	Design patterns shown
Part 1	Presentation (MVC GUI and CLI) and Facade	MVC, Facade
Part 2	Domain model and case lifecycle	State
Part 3	Log ingestion and detection	Factory Method, Strategy
Part 4	Threat intel, ATT&CK mapping and report	Adapter
Part 5	AI agent	Command, Strategy

Class Diagram Part 1: Presentation (MVC GUI and CLI) and Facade

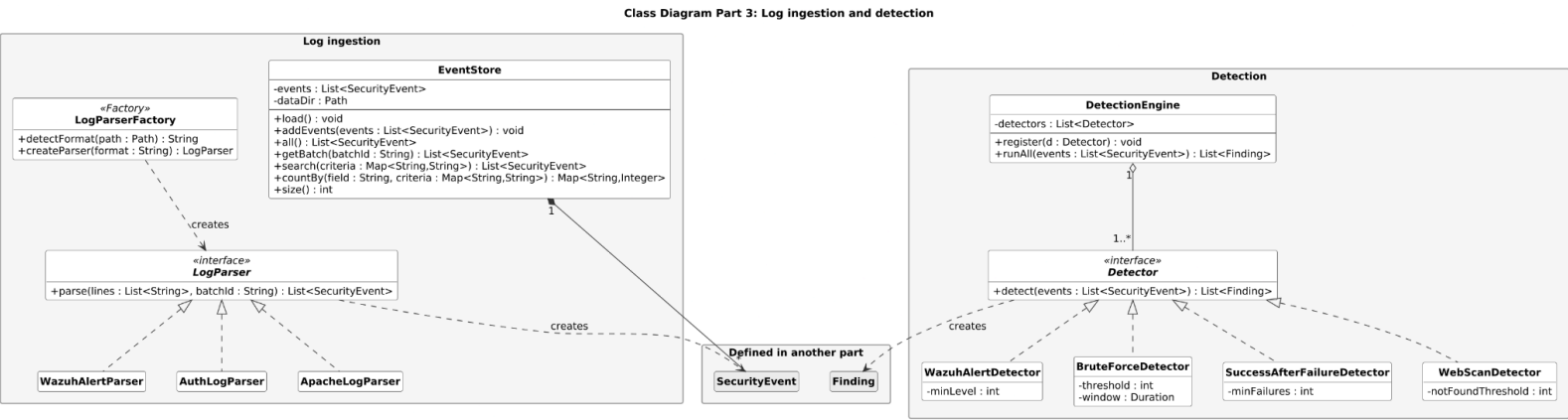


Class Diagram Part 2: Domain model and case lifecycle

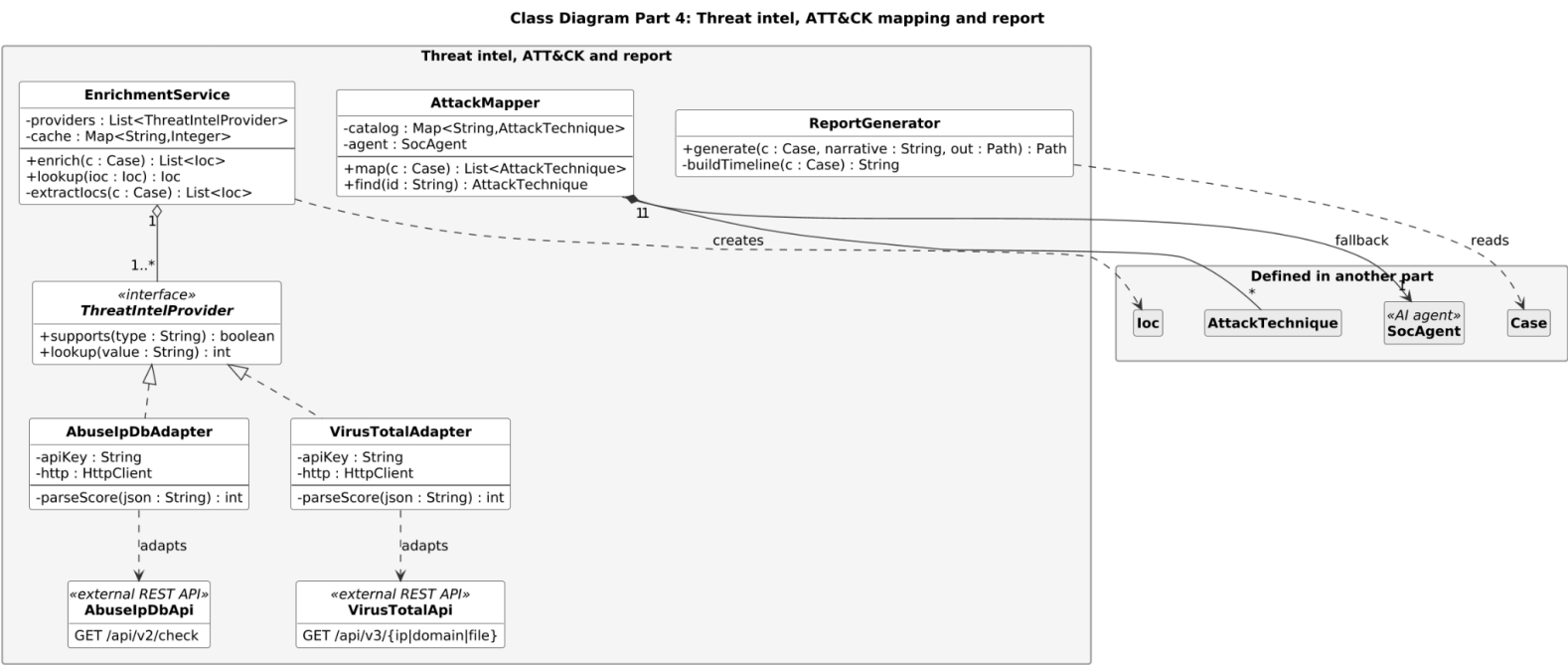
Class Diagram Part 2: Domain model and case lifecycle



Class Diagram Part 3: Log ingestion and detection

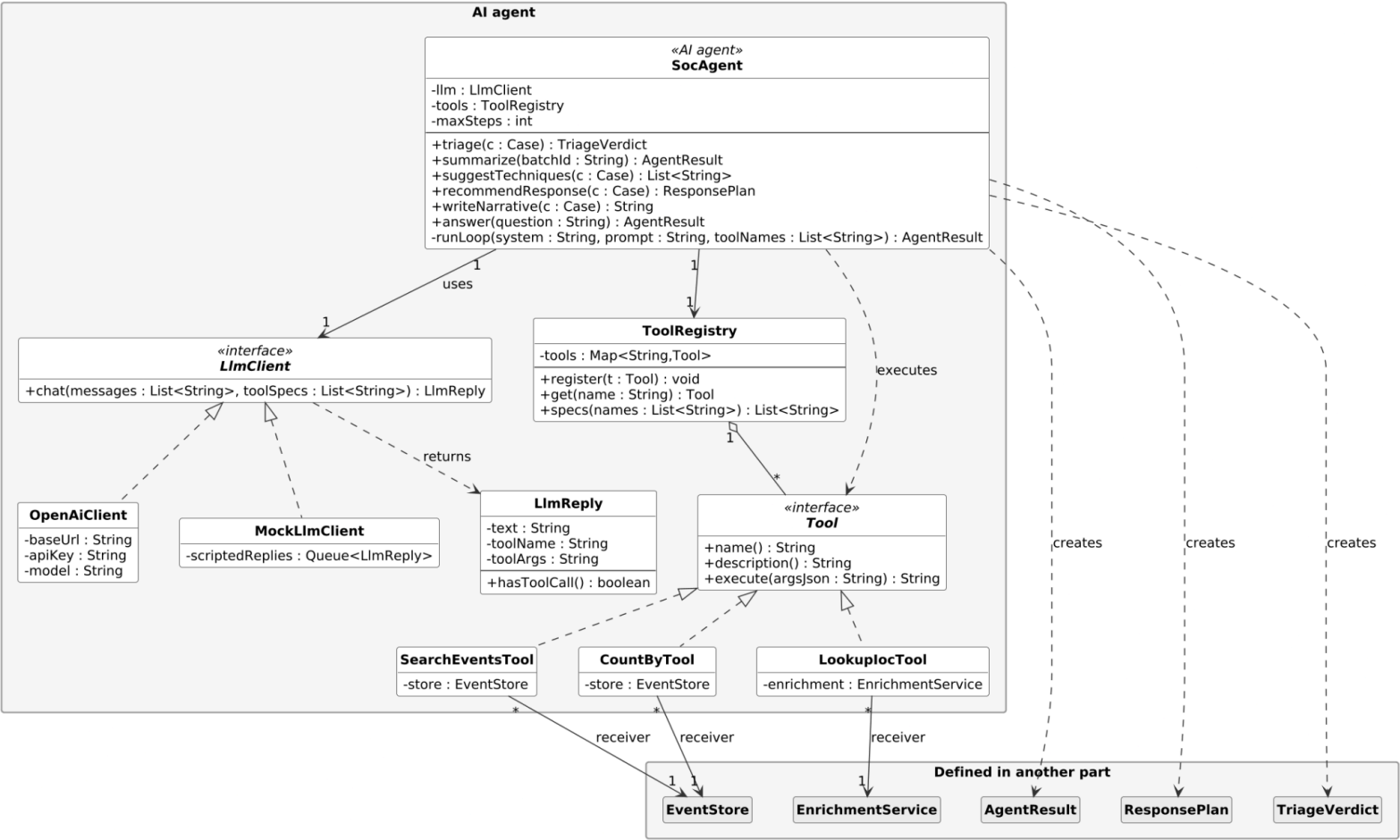


Class Diagram Part 4: Threat intel, ATT&CK mapping and report



Class Diagram Part 5: AI agent

Class Diagram Part 5: AI agent



4. Design Pattern Explanations

Seven design patterns from the course list are used. Each one solves a specific problem in TriageDesk.

4.1 Model-View-Controller (MVC)

Pattern	Model-View-Controller (MVC)
Participating classes and roles	Model: SocAssistant and the service and domain classes behind it. Views: MainView, CaseView, DashboardView, ChatView. Controllers: MainController, CaseController, DashboardController, ChatController. TriageDeskCli acts as a second view and controller for the command line.
Problem addressed	The same functions must be available from four GUI screens and from a CLI. If screens contained parsing, detection or AI logic, that logic would be duplicated and could not be tested without the GUI.
Rationale for using it	Views only display and forward user actions, controllers call the model, and the model is shared by the GUI and the CLI.
What would be harder without it	Logic would live in JavaFX event handlers, the CLI would need a second copy, and every change would have to be made twice.

4.2 Facade

Pattern	Facade
Participating classes and roles	Facade: SocAssistant. Subsystem classes: EventStore, LogParserFactory, DetectionEngine, CaseManager, EnrichmentService, AttackMapper, SocAgent, ReportGenerator. Clients: all controllers and TriageDeskCli.
Problem addressed	One user action touches several subsystems (triage uses CaseManager, SocAgent and the tools). The GUI and CLI should not need to know that wiring.
Rationale for using it	It gives both interfaces the same small set of methods, one per feature, and keeps the order of each workflow in one place.
What would be harder without it	Each controller and CLI command would depend on up to eight services, and a workflow change would have to be repeated in both interfaces.

4.3 Strategy

Pattern	Strategy
Participating classes and roles	Context: DetectionEngine; strategy: Detector; concrete strategies: WazuhAlertDetector, BruteForceDetector,

	SuccessAfterFailureDetector, WebScanDetector. Second use: context SocAgent; strategy LlmClient; concrete strategies OpenAiClient and MockLlmClient.
Problem addressed	Each detection is a different algorithm over the same events, and more will be added later. The agent must also run with a real LLM or with a scripted mock for testing and offline grading.
Rationale for using it	The algorithms are interchangeable and selected at run time from the configuration.
What would be harder without it	DetectionEngine would become a long if/else block, and SocAgent would contain HTTP code for each LLM vendor, making the AI features impossible to test without a network.

4.4 Factory Method

Pattern	Factory Method
Participating classes and roles	Creator: LogParserFactory, whose createParser(format) is a parameterized factory method (the GoF variant in which one creator returns a different product for each format chosen by detectFormat). Product: LogParser. Concrete products: WazuhAlertParser, AuthLogParser, ApacheLogParser. Client: SocAssistant.
Problem addressed	Each log format needs a different parser, and the right one is only known after reading the file.
Rationale for using it	Parser creation is kept in one class, and the client depends only on the LogParser interface.
What would be harder without it	The import code would reference every concrete parser, and adding a format would mean editing the import workflow.

4.5 Adapter

Pattern	Adapter
Participating classes and roles	Target: ThreatIntelProvider. Adapters: AbuseIpDbAdapter, VirusTotalAdapter. Adaptees: the external AbuseIPDB and VirusTotal REST APIs (AbuseIpDbApi, VirusTotalApi). Client: EnrichmentService (and LookuplocTool through it).
Problem addressed	The two services use different endpoints, authentication and JSON formats, and different scoring scales. The rest of the system needs one simple result: a score from 0 to 100.
Rationale for using it	The external APIs cannot be changed, so each one is wrapped to match the interface the application expects.
What would be harder without it	Vendor-specific JSON handling would spread into enrichment, the agent tool and the report, and replacing a provider would affect all of them.

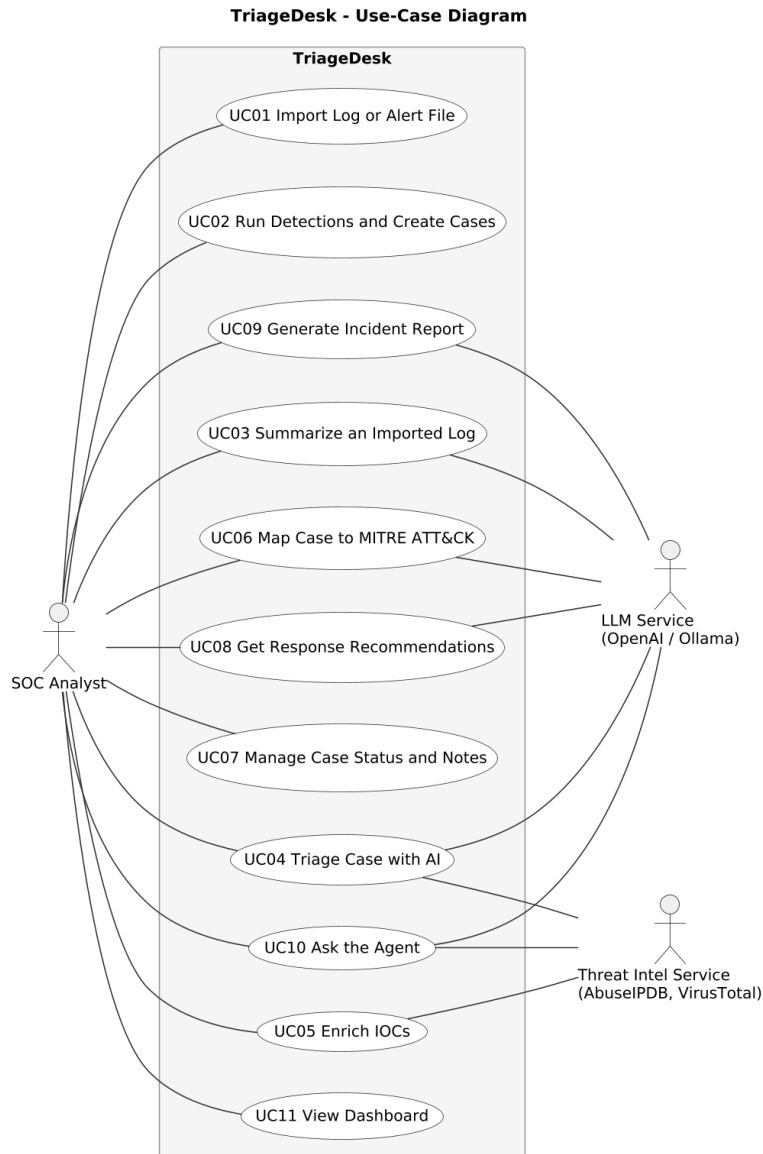
4.6 State

Pattern	State
Participating classes and roles	Context: Case. State: CaseState, an abstract class whose default methods refuse the action. Concrete states: NewState, InvestigatingState, EscalatedState, ClosedState, each overriding only the transitions it allows.
Problem addressed	The allowed actions depend on the case status: a closed case cannot be escalated, and a new case can be closed as a false positive without investigation.
Rationale for using it	Case delegates startInvestigation, escalate, close and reopen to its current state object, and each state decides the next state or refuses the action.
What would be harder without it	Every lifecycle method would repeat the same switch on a status field, and adding a new state would mean changing all of them.

4.7 Command

Pattern	Command
Participating classes and roles	Command: Tool. Concrete commands: SearchEventsTool, CountByTool, LookuplocTool. Receivers: EventStore, EnrichmentService. Invoker: SocAgent (looks tools up by name in ToolRegistry and calls execute). Client: SocAssistant, which creates each tool with its receiver and registers it in ToolRegistry at start-up; SocAgent then chooses the allowed tools for each task.
Problem addressed	The LLM decides at run time which operation to run ("call search_events with these arguments"). The agent loop must turn that request into a real method call without knowing every subsystem, and each call must be logged.
Rationale for using it	Each tool is an object that can be looked up by name, executed with JSON arguments, recorded in the tool trace and restricted per task.
What would be harder without it	The agent loop would need a switch on tool names that calls every service directly, and adding a tool would mean changing the loop.

5. Use-Case Diagram



Actors

Actor	Type	Role
SOC Analyst	Primary user	Uses all functions of the system.
LLM Service (OpenAI or Ollama)	External AI service	Provides reasoning, tool selection and text generation to the agent.
Threat Intel Service (AbuseIPDB, VirusTotal)	External API	Provides reputation data for IPs, domains and file hashes.

The eleven use cases cover all thirteen features (see the Related Feature(s) row of each description in Section 6).

6. Detailed Use-Case Descriptions

UC01: Import Log or Alert File

Use Case ID	UC01
Use Case Name	Import Log or Alert File
Actor(s)	SOC Analyst
Goal	Load a Wazuh alert export or a raw log file so it can be analysed.
Preconditions	The file exists and is readable.
Trigger	The analyst clicks Import (or runs triagedesk import).
Main Success Scenario	1. The system shows a file chooser. 2. The analyst selects a file. 3. The system detects the format. 4. The system parses each line into an event. 5. The system stores the events and reports how many were imported and skipped.
Alternative/Exception Flows	2a. The analyst cancels: nothing changes. 3a. Unsupported format: the system shows "Unsupported file format" and imports nothing. 4a. Malformed lines: skipped and counted; the import continues.
Postconditions	The events are in EventStore.
Related Feature(s)	F01, F02

UC02: Run Detections and Create Cases

Use Case ID	UC02
Use Case Name	Run Detections and Create Cases
Actor(s)	SOC Analyst
Goal	Find suspicious activity and turn it into a prioritised case list.
Preconditions	At least one file has been imported.
Trigger	The analyst clicks Run Detections (or runs triagedesk detect).
Main Success Scenario	1. The system runs every detector over the events. 2. The detectors return findings. 3. The system groups related findings into cases. 4. The system sets each case's severity and priority and saves it. 5. The case list shows the new cases by priority.
Alternative/Exception Flows	1a. No events loaded: "Import a file first". 2a. No findings: "No suspicious activity". 3a. A finding matches an open case: it is added to that case.
Postconditions	New cases exist in state New and are saved.
Related Feature(s)	F03, F05

UC03: Summarize an Imported Log

Use Case ID	UC03
Use Case Name	Summarize an Imported Log
Actor(s)	SOC Analyst, LLM Service
Goal	Get a quick plain-English overview of a log file.
Preconditions	The file has been imported; an LLM is configured.
Trigger	The analyst selects the file and clicks Summarize (or runs triagedesk summarize).
Main Success Scenario	1. The system sends a summary task to the agent, limited to that file. 2. The agent calls tools to count events by host, user and IP. 3. The agent writes the summary. 4. The system shows the summary and the tool calls.
Alternative/Exception Flows	2a. LLM unavailable: error message. 3a. Step limit reached: partial summary marked incomplete.
Postconditions	The summary is displayed.
Related Feature(s)	F04

UC04: Triage Case with AI

Use Case ID	UC04
Use Case Name	Triage Case with AI
Actor(s)	SOC Analyst, LLM Service, Threat Intel Service
Goal	Get an evidence-based verdict on whether a case is a real incident.
Preconditions	The case exists and is not closed; an LLM is configured.
Trigger	The analyst clicks AI Triage (or runs triagedesk triage).
Main Success Scenario	1. The system sends the case facts and the allowed tools to the agent. 2. The agent requests tool calls (related events, counts, IOC lookups). 3. The system runs each tool and returns the result to the agent. 4. The agent returns a verdict with confidence and reasoning. 5. The system validates and stores the verdict; a New case moves to Investigating. 6. The system shows the verdict and the tool calls.
Alternative/Exception Flows	2a. LLM unreachable: error, case unchanged. 3a. A tool fails: the error is returned to the agent, which continues. 4a. Invalid output or step limit reached: verdict NEEDS_REVIEW.
Postconditions	The case has a verdict and a record of the tool calls.
Related Feature(s)	F06

UC05: Enrich IOCs

Use Case ID	UC05
Use Case Name	Enrich IOCs
Actor(s)	SOC Analyst, Threat Intel Service
Goal	Know whether the IPs, domains and hashes in a case have a bad reputation.
Preconditions	The case exists; at least one provider API key is set.
Trigger	The analyst clicks Enrich IOCs (or runs triagedesk enrich).
Main Success Scenario	1. The system extracts the IOCs from the case's events. 2. For each IOC, the system queries each provider that supports its type. 3. The system converts each reply into a common score. 4. The system stores the results and shows the IOC table.
Alternative/Exception Flows	1a. No IOCs found: "No indicators found". 2a. Private IP: not sent externally. 2b. Timeout, rate limit or missing key: result marked unavailable; other providers continue.
Postconditions	The case's IOCs have reputation scores.
Related Feature(s)	F07

UC06: Map Case to MITRE ATT&CK

Use Case ID	UC06
Use Case Name	Map Case to MITRE ATT&CK
Actor(s)	SOC Analyst, LLM Service
Goal	Describe the case in ATT&CK terms.
Preconditions	The case exists.
Trigger	The analyst clicks Map to ATT&CK (or runs triagedesk map).
Main Success Scenario	1. The system collects the technique IDs from the case's findings. 2. The system looks each ID up in the ATT&CK catalogue. 3. The system stores and shows the techniques.
Alternative/Exception Flows	1a. No IDs on the case: the agent suggests IDs; only IDs in the catalogue are kept. 1b. No IDs and AI unavailable: "No mapping available".
Postconditions	The case lists its ATT&CK techniques.
Related Feature(s)	F08

UC07: Manage Case Status and Notes

Use Case ID	UC07
Use Case Name	Manage Case Status and Notes
Actor(s)	SOC Analyst
Goal	Move a case through its lifecycle and document the work.
Preconditions	The case exists.
Trigger	The analyst chooses a status action or adds a note (or runs triagedesk status / note).
Main Success Scenario	1. The analyst selects an action (start, escalate, close, reopen) and enters a comment. 2. The system checks that the current state allows the action. 3. The case moves to the new state and the change is recorded in its history. 4. The system saves and redisplay the case.
Alternative/Exception Flows	1a. Adding a note: the note is timestamped and saved. 1b. Empty note or missing close comment: rejected. 2a. Action not allowed: error explaining the allowed actions; case unchanged.
Postconditions	The case has its new state, history and notes.
Related Feature(s)	F09

UC08: Get Response Recommendations

Use Case ID	UC08
Use Case Name	Get Response Recommendations
Actor(s)	SOC Analyst, LLM Service
Goal	Get a clear list of containment and remediation steps.
Preconditions	The case has a triage verdict.
Trigger	The analyst clicks Recommend Response (or runs triagedesk respond).
Main Success Scenario	1. The system sends the verdict, IOCs and techniques to the agent. 2. The agent returns ordered steps with reasons. 3. The system stores the plan and shows it as a checklist. 4. The analyst ticks steps as they are done.
Alternative/Exception Flows	Pre-a. No verdict: "Run AI Triage first". 2a. False positive: a single "document and close" step. 2b. LLM unavailable: error.
Postconditions	The case has a response plan.
Related Feature(s)	F10

UC09: Generate Incident Report

Use Case ID	UC09
Use Case Name	Generate Incident Report
Actor(s)	SOC Analyst, LLM Service
Goal	Produce a shareable incident report for a case.
Preconditions	The case exists; the output folder is writable.
Trigger	The analyst clicks Generate Report (or runs triagedesk report).
Main Success Scenario	1. The agent writes the executive summary and analysis. 2. The system adds the case facts, timeline, IOCs, techniques, verdict, notes and response plan. 3. The system writes the Markdown file and shows its path.
Alternative/Exception Flows	1a. LLM unavailable: "[AI narrative unavailable]" is used and the report is still written. 3a. Write error: "Could not write report".
Postconditions	A report file exists.
Related Feature(s)	F11

UC10: Ask the Agent

Use Case ID	UC10
Use Case Name	Ask the Agent
Actor(s)	SOC Analyst, LLM Service, Threat Intel Service
Goal	Answer questions about the loaded data without writing queries.
Preconditions	At least one file has been imported.
Trigger	The analyst types a question and clicks Ask (or runs triagedesk ask).
Main Success Scenario	1. The system sends the question and the query tools to the agent. 2. The agent calls tools (for example count_by on source IPs). 3. The agent answers from the tool results. 4. The system shows the answer and the tool calls.
Alternative/Exception Flows	Pre-a. No data: "Import a file first". 2a. Step limit reached: partial answer marked incomplete. 3a. Question unrelated to the data: the agent declines.
Postconditions	The answer is shown.
Related Feature(s)	F12

UC11: View Dashboard

Use Case ID	UC11
Use Case Name	View Dashboard
Actor(s)	SOC Analyst
Goal	See the overall picture of the data and the cases.

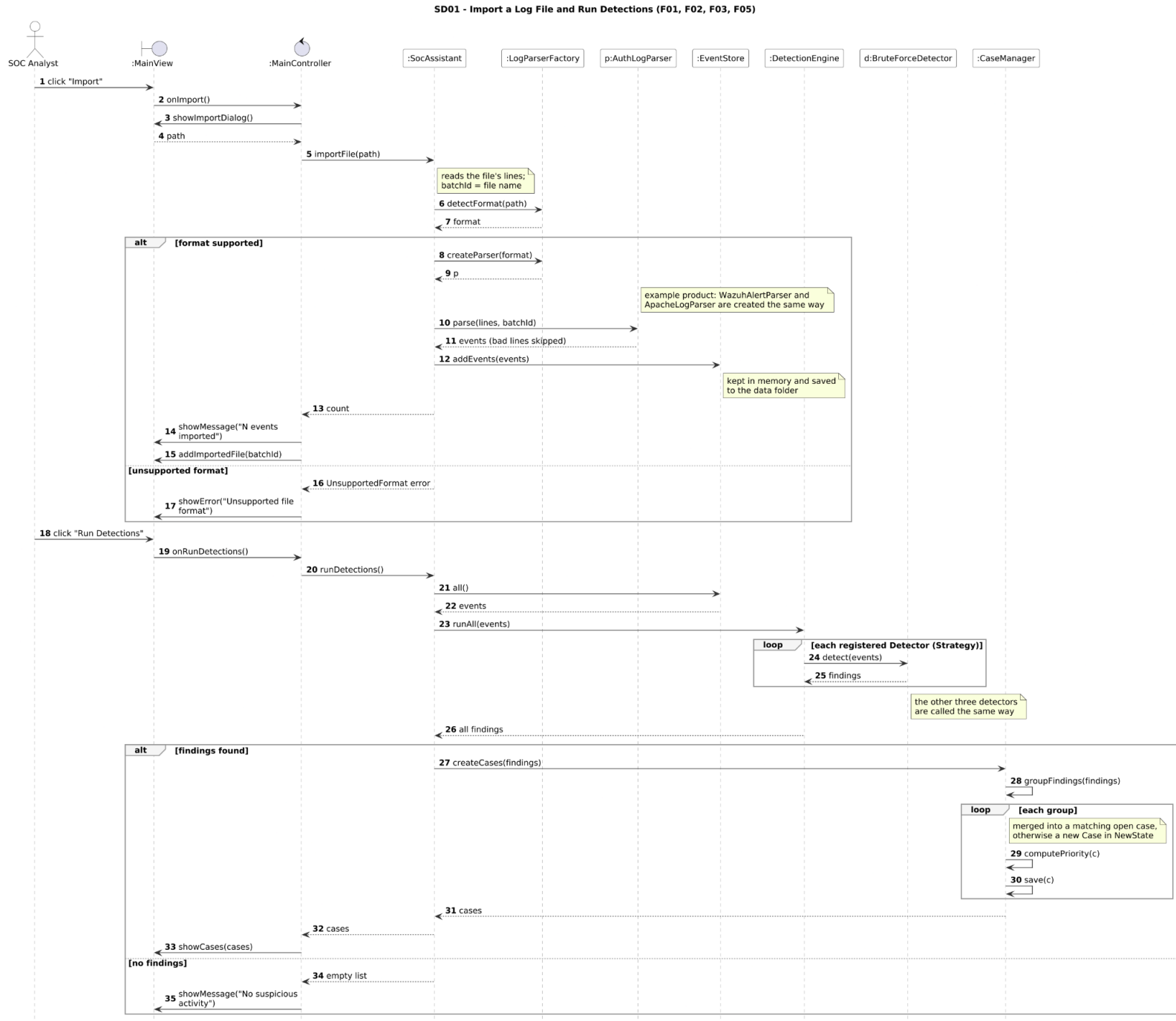
Preconditions	None.
Trigger	The analyst opens the Dashboard tab (or runs triagedesk stats).
Main Success Scenario	1. The system computes the statistics. 2. The system shows the charts and totals.
Alternative/Exception Flows	1a. No data: empty dashboard with "Import a file to begin".
Postconditions	None (read-only).
Related Feature(s)	F13

7. Sequence Diagrams

Seven sequence diagrams cover all important interactions. Features with the same interaction structure share a diagram. All objects and method calls are taken from the class diagram.

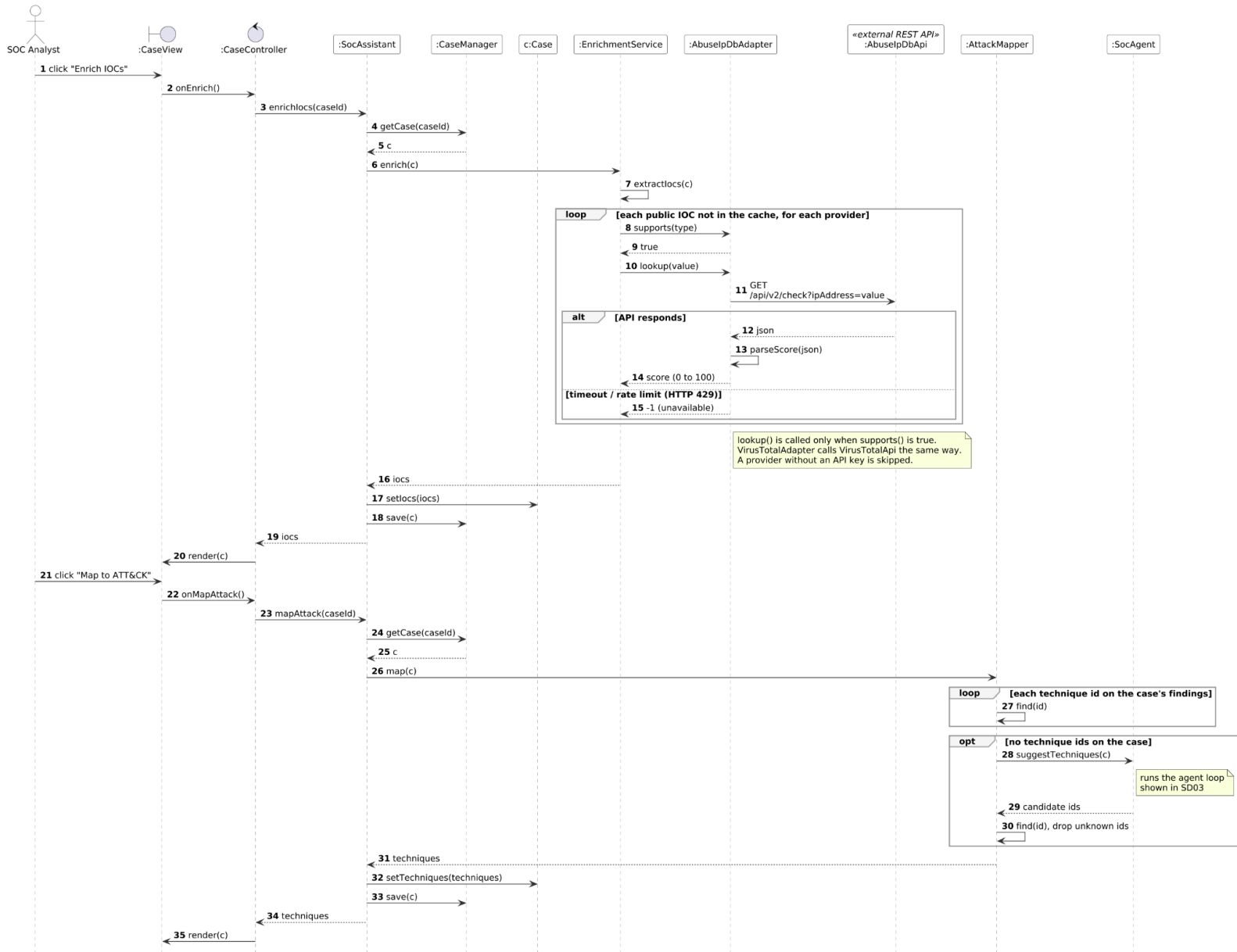
Diagram	Interaction	Features
SD01	Import a Log File and Run Detections	F01, F02, F03, F05
SD02	Enrich IOCs and Map to ATT&CK	F07, F08
SD03	AI Case Triage with Tool Use	F06
SD04	Change Case Status and Add a Note	F09
SD05	Ask the Agent from the CLI and AI Log Summary	F12, F04
SD06	Response Recommendations and Incident Report	F10, F11
SD07	Show the Dashboard	F13

SD01: Import a Log File and Run Detections (F01, F02, F03, F05)

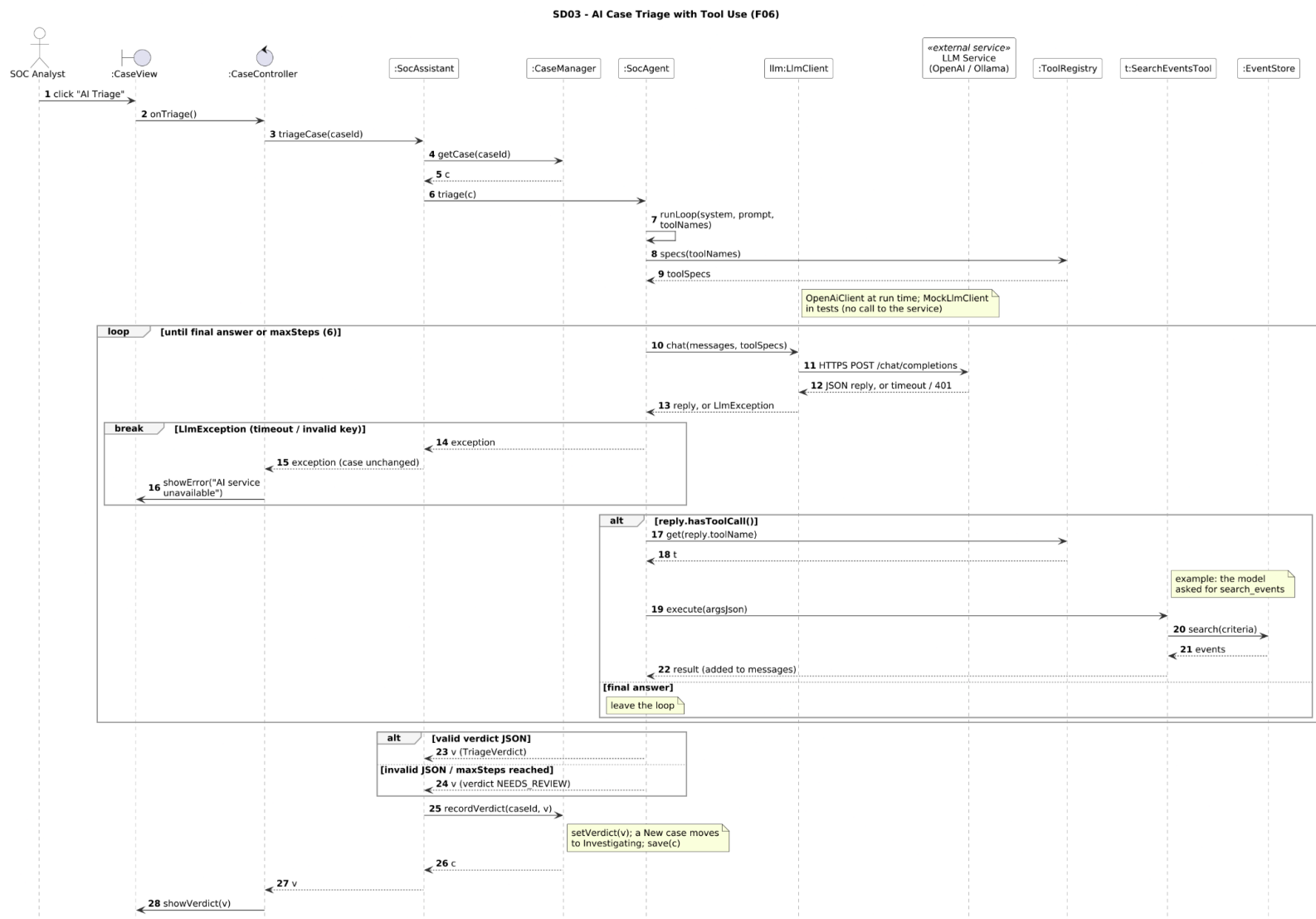


SD02: Enrich IOC and Map to ATT&CK (F07, F08)

SD02 - Enrich IOC and Map to ATT&CK (F07, F08)

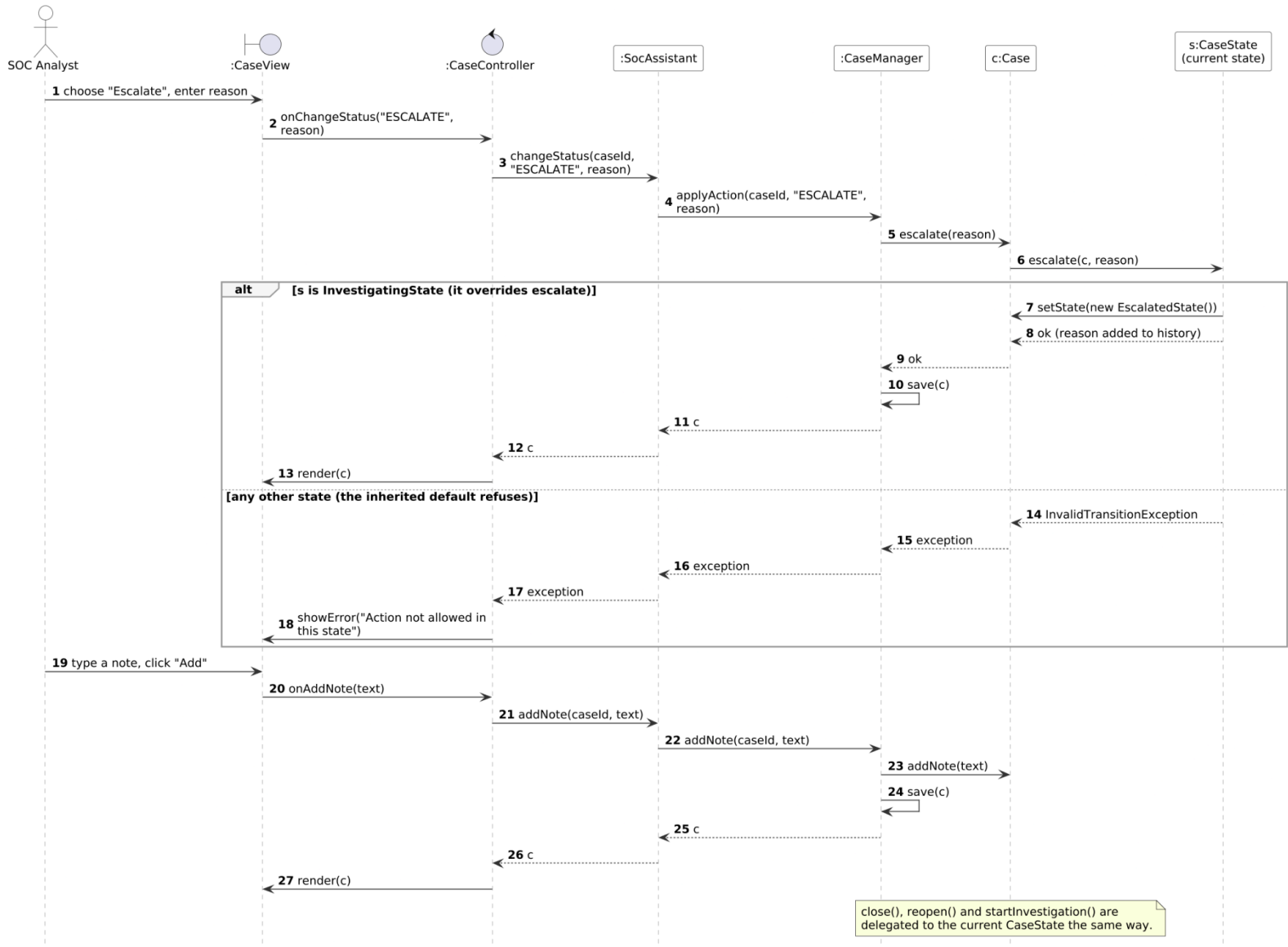


SD03: AI Case Triage with Tool Use (F06)

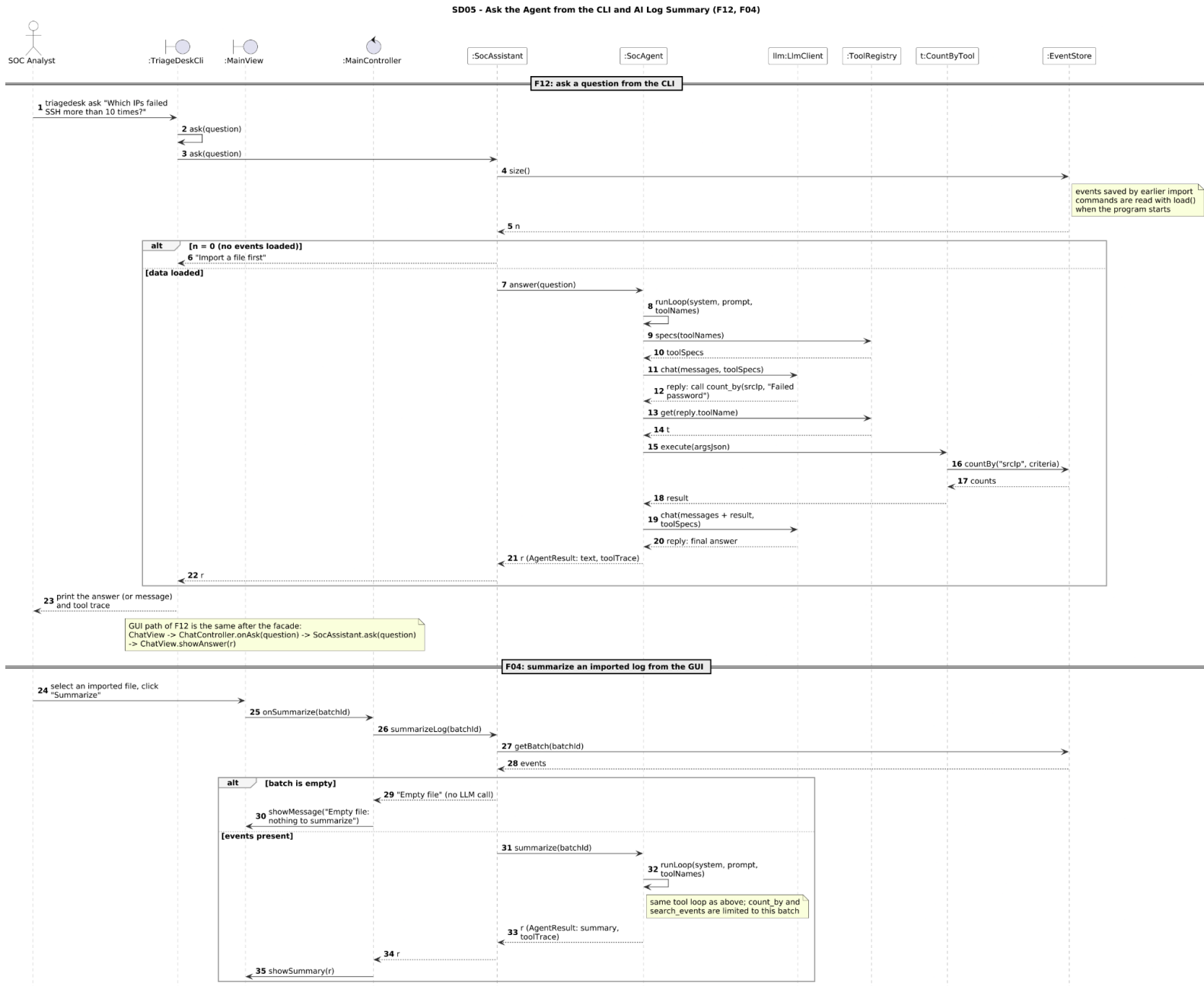


SD04: Change Case Status and Add a Note (F09)

SD04 - Change Case Status and Add a Note (F09)

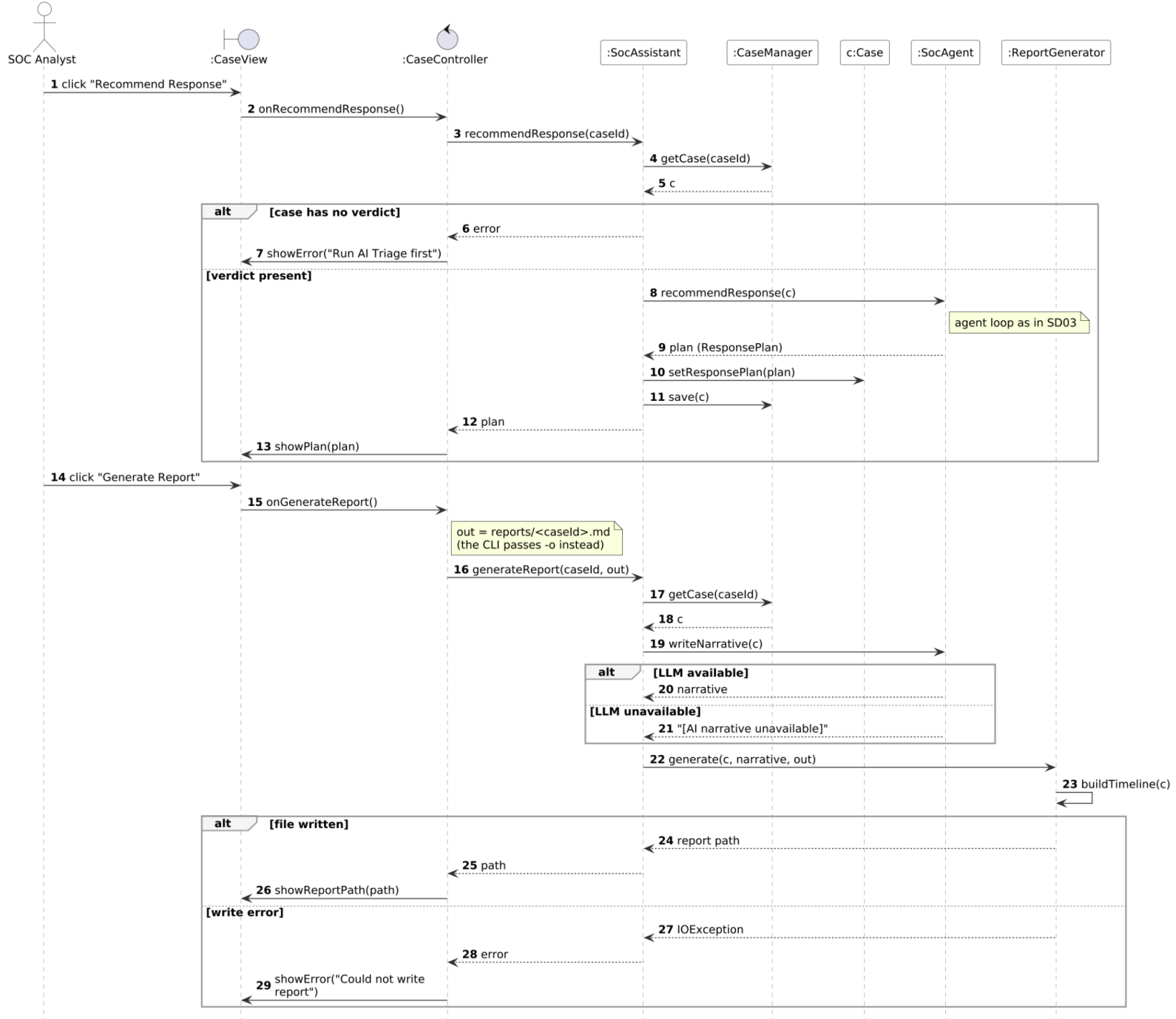


SD05: Ask the Agent from the CLI and AI Log Summary (F12, F04)



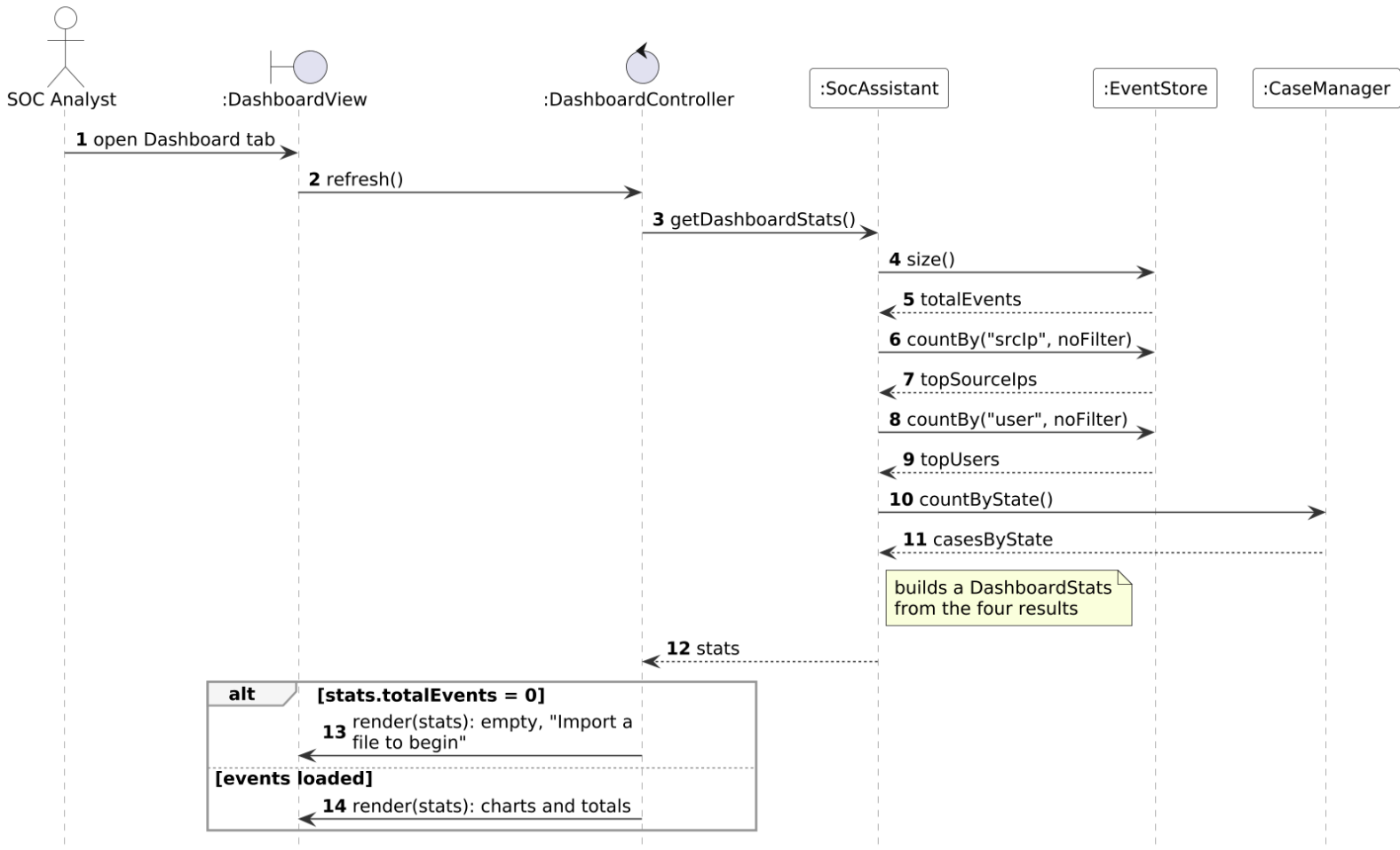
SD06: Response Recommendations and Incident Report (F10, F11)

SD06 - Response Recommendations and Incident Report (F10, F11)



SD07: Show the Dashboard (F13)

SD07 - Show the Dashboard (F13)



8. Feature-to-Design Traceability Table

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Import Wazuh Alerts	Deterministic	UC01	MainView, MainController, SocAssistant, LogParserFactory, WazuhAlertParser, EventStore	onImport(), importFile(), detectFormat(), createParser(), parse(), addEvents()	SD01	Factory Method, Facade, MVC
F02	Import and Parse Raw Logs	Deterministic	UC01	SocAssistant, LogParserFactory, AuthLogParser, ApacheLogParser, EventStore	importFile(), detectFormat(), createParser(), parse(), addEvents()	SD01	Factory Method, Facade
F03	Rule-Based Log Detections	Deterministic	UC02	MainController, SocAssistant, EventStore, DetectionEngine, Detector, WazuhAlertDetector, BruteForceDetector, SuccessAfterFailureDetector, WebScanDetector	onRunDetections(), runDetections(), all(), runAll(), detect()	SD01	Strategy, Facade
F04	AI Log Summary	AI	UC03	MainController, MainView, SocAssistant, SocAgent, LlmClient, ToolRegistry, CountByTool, SearchEventsTool, EventStore	onSummarize(), summarizeLog(), summarize(), runLoop(), chat(), execute(), showSummary()	SD05	Command, Strategy, Facade
F05	Group Findings into Prioritised Cases	Deterministic	UC02	SocAssistant, CaseManager, Case, Finding, MainView	createCases(), groupFindings(), computePriority(), save(), showCases()	SD01	Facade
F06	AI Case Triage	AI	UC04	CaseView, CaseController, SocAssistant, CaseManager, SocAgent, LlmClient, ToolRegistry, SearchEventsTool, EventStore, TriageVerdict	onTriage(), triageCase(), getCase(), triage(), runLoop(), specs(), chat(), get(), execute(), search(), recordVerdict(), showVerdict()	SD03	Command, Strategy, Facade, State
F07	IOC Extraction and Threat-Intel Enrichment	Deterministic	UC05	CaseView, CaseController, SocAssistant, CaseManager, Case, EnrichmentService, ThreatIntelProvider, AbuseIpDbAdapter, VirusTotalAdapter, loc	onEnrich(), enrichlocs(), getCase(), enrich(), extractlocs(), lookup(), parseScore(), setlocs(), save()	SD02	Adapter, Facade
F08	MITRE ATT&CK Mapping	Hybrid	UC06	CaseView, CaseController, SocAssistant, CaseManager, Case, AttackMapper, AttackTechnique, SocAgent	onMapAttack(), mapAttack(), getCase(), map(), find(), suggestTechniques(), setTechniques(), save()	SD02	Facade, Strategy
F09	Case Lifecycle Management	Deterministic	UC07	CaseView, CaseController, SocAssistant, CaseManager, Case, CaseState, NewState, InvestigatingState, EscalatedState, ClosedState	onChangeStatus(), onAddNote(), changeStatus(), applyAction(), escalate(), setState(), addNote(), save()	SD04	State, Facade, MVC

F10	AI Response Recommendations	AI	UC08	CaseView, CaseController, SocAssistant, CaseManager, Case, SocAgent, ResponsePlan	onRecommendResponse(), recommendResponse(), getCase(), setResponsePlan(), save(), showPlan()	SD06	Facade, Strategy
F11	Incident Report Generation	Hybrid	UC09	CaseView, CaseController, SocAssistant, CaseManager, SocAgent, ReportGenerator	onGenerateReport(), generateReport(), getCase(), writeNarrative(), generate(), buildTimeline(), showReportPath()	SD06	Facade
F12	Ask the Agent	AI	UC10	TriageDeskCli, ChatView, ChatController, SocAssistant, SocAgent, LlmClient, ToolRegistry, CountByTool, EventStore	ask(), onAsk(), answer(), chat(), get(), execute(), countBy(), showAnswer()	SD05	Command, Strategy, Facade, MVC
F13	Security Dashboard	Deterministic	UC11	DashboardView, DashboardController, SocAssistant, EventStore, CaseManager, DashboardStats	refresh(), getDashboardStats(), size(), countBy(), countByState(), render()	SD07	MVC, Facade

9. Feature Implementation Explanations

F01: Import Wazuh Alerts

Related Use Case: UC01, Import Log or Alert File

Related Sequence Diagram: SD01, Import a Log File and Run Detections

Classes involved:

- **MainView**: shows the file dialog, the result message and the list of imported files
- **MainController**: handles the Import button
- **SocAssistant**: facade entry point for the import
- **LogParserFactory**: detects the format and creates the parser
- **WazuhAlertParser**: converts JSON alert lines into SecurityEvents
- **EventStore**: keeps the imported events in memory and saves them to the data folder

Important methods:

- **MainController.onImport()**
- **SocAssistant.importFile()**
- **LogParserFactory.detectFormat()**
- **LogParserFactory.createParser()**
- **WazuhAlertParser.parse()**
- **EventStore.addEvents()**

Execution: When the analyst clicks Import, **MainController.onImport()** gets the path from **MainView** and calls **SocAssistant.importFile()**. The facade asks **LogParserFactory.detectFormat()** for the format and **createParser()** for a **WazuhAlertParser**, calls **parse()** on the file's lines, and stores the result with **EventStore.addEvents()**, which also saves the events to the data folder so that later CLI commands can load them. The number of events is returned and shown in **MainView**, which adds the file to its list of imported files (the file name is the batch ID used by F04).

F02: Import and Parse Raw Logs

Related Use Case: UC01, Import Log or Alert File

Related Sequence Diagram: SD01, Import a Log File and Run Detections

Classes involved:

- **SocAssistant**: facade entry point for the import
- **LogParserFactory**: chooses the parser for the detected format
- **AuthLogParser**: parses sshd and sudo lines
- **ApacheLogParser**: parses access-log lines (IP, time, path, status)
- **EventStore**: keeps the events

Important methods:

- **SocAssistant.importFile()**
- **LogParserFactory.detectFormat()**

- LogParserFactory.createParser()
- AuthLogParser.parse()
- EventStore.addEvents()

Execution: The flow is the same as F01. The only difference is the concrete parser returned by LogParserFactory.createParser(), which is why the factory is used: SocAssistant only depends on the LogParser interface.

F03: Rule-Based Log Detections

Related Use Case: UC02, Run Detections and Create Cases

Related Sequence Diagram: SD01, Import a Log File and Run Detections

Classes involved:

- MainController: handles the Run Detections button
- SocAssistant: facade entry point
- DetectionEngine: runs every registered detector
- Detector: strategy interface for one detection rule
- BruteForceDetector: example strategy: counts failed logins per IP in a time window

Important methods:

- MainController.onRunDetections()
- SocAssistant.runDetections()
- EventStore.all()
- DetectionEngine.runAll()
- Detector.detect()

Execution: SocAssistant.runDetections() gets all events from EventStore.all() and passes them to DetectionEngine.runAll(), which calls detect() on each Detector. For example, BruteForceDetector groups failed-login events by source IP and creates a HIGH-severity Finding with technique T1110.001 when the count in 5 minutes reaches the threshold. The findings are returned to the facade for F05.

F04: AI Log Summary

Related Use Case: UC03, Summarize an Imported Log

Related Sequence Diagram: SD05, Ask the Agent from the CLI and AI Log Summary

Classes involved:

- MainController: handles the Summarize button
- SocAssistant: facade entry point
- SocAgent: builds the summary task and runs the agent loop
- LlmClient: sends the conversation to the model
- CountByTool: counts events by field for the agent
- SearchEventsTool: returns sample events for the agent

- **MainView:** shows the summary

Important methods:

- `MainController.onSummarize()`
- `SocAssistant.summarizeLog()`
- `SocAgent.summarize()`
- `SocAgent.runLoop()`
- `LlmClient.chat()`
- `CountByTool.execute()`
- `MainView.showSummary()`

Execution: `MainController.onSummarize()` calls `SocAssistant.summarizeLog(batchId)`, which calls `SocAgent.summarize(batchId)`. The agent runs `runLoop()` with the `count_by` and `search_events` tools restricted to that batch: the model asks for counts of hosts, users and IPs, the tools query `EventStore`, and the model writes the summary. The result is returned as an `AgentResult` and shown with `MainView.showSummary()`.

F05: Group Findings into Prioritised Cases

Related Use Case: UC02, Run Detections and Create Cases

Related Sequence Diagram: SD01, Import a Log File and Run Detections

Classes involved:

- **SocAssistant:** passes the findings to `CaseManager`
- **CaseManager:** groups findings, creates and saves cases
- **Case:** the created case
- **Finding:** the input of the grouping
- **MainView:** shows the case list

Important methods:

- `CaseManager.createCases()`
- `CaseManager.groupFindings()`
- `CaseManager.computePriority()`
- `CaseManager.save()`
- `MainView.showCases()`

Execution: `SocAssistant` passes the findings to `CaseManager.createCases()`. `groupFindings()` keys each finding by source IP (or host) and merges findings less than 60 minutes apart. Each group becomes a `Case` in `NewState`; `computePriority()` sets its priority from the severity and number of findings, and `save()` writes it to a JSON file. The cases are returned and shown with `MainView.showCases()`.

F06: AI Case Triage

Related Use Case: UC04, Triage Case with AI

Related Sequence Diagram: SD03, AI Case Triage with Tool Use

Classes involved:

- CaseController: handles the AI Triage button
- SocAssistant: coordinates the triage
- CaseManager: loads the case and records the verdict
- SocAgent: builds the triage prompt and runs the agent loop
- LlmClient: sends the conversation to the model (OpenAiClient at run time)
- ToolRegistry: finds a tool by name
- SearchEventsTool: searches EventStore for related events
- TriageVerdict: the result
- CaseView: shows the verdict

Important methods:

- CaseController.onTriage()
- SocAssistant.triageCase()
- CaseManager.getCase()
- SocAgent.triage()
- SocAgent.runLoop()
- ToolRegistry.specs()
- LlmClient.chat()
- ToolRegistry.get()
- SearchEventsTool.execute()
- EventStore.search()
- CaseManager.recordVerdict()
- CaseView.showVerdict()

Execution: CaseController.onTriage() calls SocAssistant.triageCase(), which loads the case with CaseManager.getCase() and calls SocAgent.triage(). The agent starts runLoop() with the case facts and the tool list from ToolRegistry.specs(). Each loop step calls chat() on the LlmClient. If the reply asks for a tool, the agent gets it with ToolRegistry.get() and calls execute(); for example SearchEventsTool calls EventStore.search() for events on the same host. The result goes back to the model. When the model returns the final JSON, the agent parses it into a TriageVerdict (NEEDS_REVIEW if invalid). SocAssistant passes it to CaseManager.recordVerdict(), which stores the verdict, moves a New case to Investigating and saves it; CaseView.showVerdict() displays it.

F07: IOC Extraction and Threat-Intel Enrichment

Related Use Case: UC05, Enrich IOCs

Related Sequence Diagram: SD02, Enrich IOCs and Map to ATT&CK

Classes involved:

- CaseController: handles the Enrich IOCs button
- EnrichmentService: extracts IOCs, calls providers, caches results
- ThreatIntelProvider: common interface for all providers
- AbuseIpDbAdapter: adapts the AbuseIPDB API
- VirusTotalAdapter: adapts the VirusTotal API
- loc: an indicator with its score

Important methods:

- CaseController.onEnrich()
- SocAssistant.enrichlocs()
- CaseManager.getCase()
- EnrichmentService.enrich()
- EnrichmentService.extractlocs()
- ThreatIntelProvider.lookup()
- AbuseIpDbAdapter.parseScore()
- Case.setlocs()
- CaseManager.save()

Execution: SocAssistant.enrichlocs() loads the case and calls EnrichmentService.enrich(). extractlocs() finds the indicators; for each one not in the cache, the service calls lookup(value) on every ThreatIntelProvider that supports its type. Each adapter calls its vendor's REST API and converts the reply with parseScore() into a 0 to 100 score. Failures return "unavailable" instead of an exception. SocAssistant stores the IOCs with Case.setlocs() and saves the case with CaseManager.save().

F08: MITRE ATT&CK Mapping

Related Use Case: UC06, Map Case to MITRE ATT&CK

Related Sequence Diagram: SD02, Enrich IOCs and Map to ATT&CK

Classes involved:

- CaseController: handles the Map button
- AttackMapper: resolves technique IDs against the catalogue
- SocAgent: suggests IDs when the case has none
- AttackTechnique: a resolved technique

Important methods:

- CaseController.onMapAttack()
- SocAssistant.mapAttack()
- CaseManager.getCase()
- AttackMapper.map()

- AttackMapper.find()
- SocAgent.suggestTechniques()
- Case.setTechniques()
- CaseManager.save()

Execution: SocAssistant.mapAttack() calls AttackMapper.map(c), which calls find(id) for each technique ID already on the case's findings. Only if there are none does it call SocAgent.suggestTechniques(c); each suggested ID is checked with find() and unknown IDs are dropped. SocAssistant stores the techniques with Case.setTechniques() and saves the case.

F09: Case Lifecycle Management

Related Use Case: UC07, Manage Case Status and Notes

Related Sequence Diagram: SD04, Change Case Status and Add a Note

Classes involved:

- CaseController: handles status buttons and notes
- CaseManager: applies the action and saves the case
- Case: context that delegates to its current state
- CaseState: abstract state; refuses every action unless a subclass overrides it
- InvestigatingState: example state that allows escalation

Important methods:

- CaseController.onChangeStatus()
- CaseController.onAddNote()
- SocAssistant.changeStatus()
- CaseManager.applyAction()
- Case.escalate()
- Case.setState()
- CaseManager.addNote()
- CaseManager.save()

Execution: CaseController.onChangeStatus() calls SocAssistant.changeStatus(), which calls CaseManager.applyAction(). The manager calls the matching Case method, for example escalate(reason). Case forwards it to its current CaseState: InvestigatingState.escalate() records the reason and calls setState(new EscalatedState()), while ClosedState does not override escalate(), so the inherited default throws an exception. On success the case is saved and CaseView redisplay it. Notes follow the path addNote() then save().

F10: AI Response Recommendations

Related Use Case: UC08, Get Response Recommendations

Related Sequence Diagram: SD06, Response Recommendations and Incident Report

Classes involved:

- CaseController: handles the button
- SocAssistant: checks the verdict and stores the plan
- SocAgent: asks the LLM for the steps
- ResponsePlan: the checklist
- CaseView: shows the checklist

Important methods:

- CaseController.onRecommendResponse()
- SocAssistant.recommendResponse()
- CaseManager.getCase()
- Case.setResponsePlan()
- CaseManager.save()
- CaseView.showPlan()

Execution: SocAssistant.recommendResponse() loads the case with CaseManager.getCase() and refuses it if there is no verdict. Otherwise it calls SocAgent.recommendResponse(c), which runs the agent loop with the verdict, IOCs and techniques and parses the returned steps into a ResponsePlan. The plan is stored with Case.setResponsePlan(), saved, and shown with CaseView.showPlan().

F11: Incident Report Generation

Related Use Case: UC09, Generate Incident Report

Related Sequence Diagram: SD06, Response Recommendations and Incident Report

Classes involved:

- CaseController: handles the button
- SocAssistant: coordinates narrative and report
- SocAgent: writes the narrative
- ReportGenerator: builds and writes the report

Important methods:

- CaseController.onGenerateReport()
- SocAssistant.generateReport()
- CaseManager.getCase()
- SocAgent.writeNarrative()
- ReportGenerator.generate()
- ReportGenerator.buildTimeline()
- CaseView.showReportPath()

Execution: SocAssistant.generateReport() loads the case with CaseManager.getCase() and asks SocAgent.writeNarrative(c) for the summary text (or uses a placeholder if the LLM fails), then calls ReportGenerator.generate(c, narrative, out). The generator builds the timeline with buildTimeline(), adds the case sections, writes the file and returns its path, which CaseView.showReportPath() displays.

F12: Ask the Agent

Related Use Case: UC10, Ask the Agent

Related Sequence Diagram: SD05, Ask the Agent from the CLI and AI Log Summary

Classes involved:

- TriageDeskCli: CLI entry point (ask command)
- ChatController: GUI entry point
- SocAgent: runs the agent loop
- CountByTool: counts events by a field
- EventStore: the data queried
- ChatView: shows the answer in the GUI

Important methods:

- TriageDeskCli.ask()
- ChatController.onAsk()
- SocAgent.answer()
- LlmClient.chat()
- ToolRegistry.get()
- CountByTool.execute()
- EventStore.countBy()
- ChatView.showAnswer()

Execution: TriageDeskCli.ask() (or ChatController.onAsk()) calls SocAssistant.ask(question), which checks that data is loaded and calls SocAgent.answer(). The model asks for count_by on srcIp filtered on failed logins; the agent gets CountByTool from ToolRegistry and calls execute(), which calls EventStore.countBy(). The model then writes the answer from the counts. The CLI prints the answer and tool trace; the GUI shows it with ChatView.showAnswer().

F13: Security Dashboard

Related Use Case: UC11, View Dashboard

Related Sequence Diagram: SD07, Show the Dashboard

Classes involved:

- DashboardController: refreshes the dashboard
- SocAssistant: collects the statistics
- EventStore: event counts
- CaseManager: case counts by state
- DashboardView: draws the charts

Important methods:

- DashboardController.refresh()
- SocAssistant.getDashboardStats()

- `EventStore.size()`
- `EventStore.countBy()`
- `CaseManager.countByState()`
- `DashboardView.render()`

Execution: When the Dashboard tab opens, `DashboardController.refresh()` calls `SocAssistant.getDashboardStats()`. The facade uses `EventStore.size()` and `countBy()` for top IPs and users, and `CaseManager.countByState()` for the case counts, and returns a `DashboardStats` object that `DashboardView.render()` draws.